

Dato de Referencia MIPS32

Renny Andrade

May 2026

1. **add:**

Suma 2 registros y guarda el resultado en otro registro aparte.

Ejemplo:

```
add $t0, $t1, $t2    # $t0 = $t1 + $t2
```

2. **addi:**

Suma 1 registro y un valor constante y guarda el resultado en otro registro aparte.

Ejemplo:

```
addi $t0, $t1, 10    # $t0 = $t1 + 10
```

3. **sub:**

Resta 2 registros y guarda el resultado en otro registro aparte.

Ejemplo:

```
sub $t0, $t1, $t2    # $t0 = $t1 - $t2
```

4. **la (Load Address):**

Guardamos la dirección de memoria de una variable a un registro.

5. **lw (Load Word):**

Guardamos el valor que tiene una variable a un registro.

6. sw (Store Word):

Guardamos el valor que tiene un registro a una variable.

Ejemplo:

```
.data
numero: .word 10      # creamos una variable numero con un 10

.text
main:
    la $t0, numero    # guarda la dirección de memoria de "numero"
    lw $t1, 0($t0)    # guardamos el 10 en el registro $t1 ($t1 = 10)
    addi $t2, $t1, 40 # $t2 = $t1 + 40 = 10 + 40 = 50
    sw $t2, numero    # guarda el valor de $t2 en numero (numero = 50)
```

Un `.word` mide 4 bytes. Puede darse el caso en el que una variable tenga un conjunto de números. Para guardar o leer con `lw` y `sw`, se debe ir sumando de 4 en 4:

Ejemplo:

```
.data
numeros: .word 10, 20, 30, 40 # Array de palabras

.text
main:
    la $t0, numeros    # dirección base del array
    lw $t1, 0($t0)     # $t1 = 10
    lw $t2, 4($t0)     # $t2 = 20
    lw $t3, 8($t0)     # $t3 = 30
    lw $t4, 12($t0)    # $t4 = 40
```

Existen variantes del `lw` y `sw`:

- `lh` y `sh` hacen las mismas funciones que `lw` y `sw` pero manejan solo 2 bytes en la RAM (Halfword).

Ejemplo con lh:

```
lh $t1, 0($t0)    # $t1 = 10
lh $t2, 2($t0)    # $t2 = 20
lh $t3, 4($t0)    # $t3 = 30
lh $t4, 6($t0)    # $t4 = 40
```

- `lb` y `sb` hacen las mismas funciones que `lw` y `sw` pero manejan solo 1 byte en la RAM.

Ejemplo con lb:

```
lb $t1, 0($t0)    # $t1 = 10
lb $t2, 1($t0)    # $t2 = 20
lb $t3, 2($t0)    # $t3 = 30
lb $t4, 3($t0)    # $t4 = 40
```

7. srl (Shift Right Logical):

Mueve los bits del registro hacia la derecha. Podemos indicarles cuántos espacios debe desplazarse.

Ejemplo:

```
# $t0 = 12 = 00001100
srl $t1, $t0, 2    # $t1 = 00000011 = 3
```

8. sll (Shift Left Logical):

Mueve los bits del registro hacia la izquierda. Podemos indicarles cuántos espacios debe desplazarse.

Ejemplo:

```
# $t0 = 3 = 00000011
sll $t1, $t0, 2    # $t1 = 00001100 = 12
```

9. and:

```
and $t0, $t1, $t2 # $t0 = $t1 & $t2 (Bitwise AND)
```

10. or:

```
or $t0, $t1, $t2  # $t0 = $t1 | $t2 (Bitwise OR)
```

11. not:

Esta función invierte los bits de un valor (pasa los 0 a 1 y viceversa). Sin embargo, en caso de que se desee tener el negativo de un número, no servirá hacer solo esto. Para poder tener el negativo de un número primero debemos invertir los bits (complemento a 1) y luego sumarle un bit (complemento a 2).

```
not $t0, $t1      # $t0 = ~($t1)
addi $t0, $t0, 1  # $t0 = -$t1
```

12. beq (Branch if Equal):

Compara si 2 registros son iguales. En caso de cumplirse, saltará a la etiqueta indicada.

Ejemplo:

```
beq $t0, $t1, etiqueta # si $t0 == $t1 saltará a "etiqueta"
                        (salto a beq otra vez)
etiqueta:
...
```

13. bne (Branch if Not Equal):

Compara si 2 registros son distintos. En caso de cumplirse, saltará a la etiqueta indicada.

Ejemplo:

```
bne $t0, $t1, etiqueta # si $t0 != $t1 saltará a "etiqueta"
    (salto a bne otra vez)
etiqueta:
    ...
```

14. bgt (Branch if Greater Than):

Compara si un registro es mayor que otro. En caso de cumplirse, saltará a la etiqueta indicada.

Ejemplo:

```
bgt $t0, $t1, etiqueta # si $t0 > $t1 saltará a "etiqueta"
    (salto a bgt otra vez)
etiqueta:
    ...
```

15. bge (Branch if Greater or Equal):

Compara si un registro es mayor o igual que otro. En caso de cumplirse, saltará a la etiqueta indicada.

Ejemplo:

```
bge $t0, $t1, etiqueta # si $t0 >= $t1 saltará a "etiqueta"
    (salto a bge otra vez)
etiqueta:
    ...
```

16. bgtz (Branch if Greater Than Zero):

Compara si un registro es mayor que 0. En caso de cumplirse, saltará a la etiqueta indicada.

Ejemplo:

```
bgtz $t0, etiqueta    # si $t0 > 0 saltará a "etiqueta"
    (salto a bgtz otra vez)
etiqueta:
    ...
```

17. bgez (Branch if Greater or Equal Than Zero):

Compara si un registro es mayor o igual a 0. En caso de cumplirse, saltará a la etiqueta indicada.

Ejemplo:

```
bgez $t0, etiqueta      # si $t0 >= 0 saltará a "etiqueta"
    (salto a bgez otra vez)
etiqueta:
    ...
```

18. ble (Branch if Less or Equal):

Compara si un registro es menor o igual a otro. En caso de cumplirse, saltará a la etiqueta indicada.

Ejemplo:

```
ble $t0, $t1, etiqueta # si $t0 <= $t1 saltará a "etiqueta"
    (salto a ble otra vez)
etiqueta:
    ...
```

19. blt (Branch if Less Than):

Compara si un registro es menor que otro. En caso de cumplirse, saltará a la etiqueta indicada.

Ejemplo:

```
blt $t0, $t1, etiqueta # si $t0 < $t1 saltará a "etiqueta"
    (salto a blt otra vez)
etiqueta:
    ...
```

20. blez (Branch if Less or Equal Zero):

Compara si un registro es menor o igual a 0. En caso de cumplirse, saltará a la etiqueta indicada.

Ejemplo:

```
blez $t0, etiqueta      # si $t0 <= 0 saltará a "etiqueta"
    (salto a blez otra vez)
etiqueta:
    ...
```

21. bltz (Branch if Less Than Zero):

Compara si un registro es menor a 0. En caso de cumplirse, saltará a la etiqueta indicada.

Ejemplo:

```
bltz $t0, etiqueta      # si $t0 < 0 saltará a "etiqueta"
                        (salto a bltz otra vez)
etiqueta:
    ...
```

22. slt (Set on Less Than):

Compara si un registro es menor que otro. No dirige a otra etiqueta en caso de cumplirse la condición, solo guarda el resultado en el registro destino (1 si es verdadero, 0 si es falso). *(Nota: Corregido el comentario original, ya que slt evalúa "menor que").*

Ejemplo:

```
slt $t0, $t1, $t2      # $t0 = 1 si ($t1 < $t2), de lo contrario $t0 = 0
```

23. j (Jump):

Salta a una etiqueta asignada de forma incondicional.

Ejemplo (Ciclo While):

```
# Suponiendo que $t0 = 0 y $t1 = 10
while:                                # etiqueta que indica el inicio del bucle
    bgt $t0, $t1, continuacion # si $t0 > $t1 sale del ciclo
    addi $t0, $t0, 1           # incrementamos $t0
    j while                    # regresamos al inicio del ciclo
continuation:
    ...
```